

Data Structures

CSC 301

Lecture Notes

Chapter 4: Stacks, Queues, and Linked Structures

Computer Science
Osun State University
2026/2027

Prepared by
Tunde Sardine

Contents

4	Stacks, Queues, and Linked Structures	2
4.1	Introduction	2
4.1.1	Stacks	2
4.1.2	Queues	2
4.1.3	Linked Structures	2
4.1.4	Implementation Strategies for Stacks and Queues	3
4.1.5	Comparison of Stack and Queue Implementations	3
4.1.6	Conclusion	3

Chapter 4

Stacks, Queues, and Linked Structures

4.1 Introduction

Stacks and queues are abstract data types that model collection behaviors with specific ordering constraints. A stack follows the last-in, first-out (LIFO) principle, while a queue follows the first-in, first-out (FIFO) principle. These structures are fundamental for managing task scheduling, expression evaluation, and breadth-first traversal in algorithms.

4.1.1 Stacks

A stack is a linear collection that supports two primary operations: **push**, which adds an element to the top, and **pop**, which removes the top element. The most recent element inserted is the first one removed, adhering to the LIFO order. Stacks may be implemented using an array or a linked list, each with distinct memory characteristics.

4.1.2 Queues

A queue is a linear collection that supports **enqueue**, which adds an element to the rear, and **dequeue**, which removes the front element. The earliest element inserted is the first one removed, satisfying the FIFO order. Like stacks, queues can be realized with contiguous arrays or with linked structures.

4.1.3 Linked Structures

Linked structures consist of **nodes** connected by **pointers** or **references**. Each node typically stores data and a link to another node, allowing the overall structure to grow dynamically without requiring contiguous memory. This contrasts with arrays, where elements are stored in adjacent memory locations.

4.1.4 Implementation Strategies for Stacks and Queues

Two primary implementation strategies exist:

- **Array-based implementation:** Memory is allocated as a contiguous block. Push and pop (or enqueue and dequeue) operations are $O(1)$ when the structure does not exceed capacity, but resizing may be required, leading to occasional $O(n)$ cost.
- **Linked-list-based implementation:** Each element is a node allocated on the heap. Insertions and deletions are $O(1)$ without resizing, but each operation incurs additional overhead for pointer manipulation and memory allocation.

The choice between these strategies affects time complexity, memory usage, and flexibility, especially when the size of the collection is not known in advance.

4.1.5 Comparison of Stack and Queue Implementations

Arrays provide constant-time random access, which benefits stack **push** and **pop** operations, but they limit dynamic resizing. Linked lists eliminate the need for resizing and allow unlimited growth, at the cost of extra memory for pointers and slower traversal. Queues implemented with a circular buffer (array-based) achieve $O(1)$ enqueue and dequeue, while a linked-list queue maintains $O(1)$ operations without the complexity of buffer wrap-around logic.

4.1.6 Conclusion

Stacks and queues are essential abstract data types whose implementations can be tailored to application requirements. Array-based structures offer speed and simplicity for fixed-size scenarios, whereas linked-list-based structures provide dynamic sizing and flexibility at the expense of additional overhead. Understanding these trade-offs is crucial for effective data structure selection in system design.

Tutorial Questions

1. What principle does a stack follow, and how is it formally expressed?
2. Explain the difference between array-based and linked-list-based implementations of a stack.
3. How does the memory access pattern of a queue differ from that of a stack?
4. If a stack initially empty performs the operations **push 10**, **push 20**, **pop**, **push 30**, what is the value on the top of the stack after the final operation?
5. Define the term **node** as used in linked structures.
6. Compare the time complexity of **enqueue** and **dequeue** operations in an array-based queue versus a linked-list-based queue.

7. What role does **memory alignment** play in the layout of a structure containing both an array and a pointer?
8. Explain why a queue implemented with a linked list may avoid the need for resizing operations.